The first two notebooks are nearly identical to their counterparts from the last chapter, so we won't cover them again. We start from the `CH5-03-FE Using Spark Structured Streaming` notebook.

Building a streaming feature with Spark Structured Streaming

We calculate the number of transactions per customer in the last two minutes and call this new feature `transactionCount`, which is a case class in the code.

CustomerID	transactionTimestamp	…	…
1	2023-09-03 19:23:09.765676		
4	2023-09-03 19:23:09.765821		
2	2023-09-03 19:23:09.765899		

Figure 5.6 – The table contains example data corresponding to readStream – namely, inputDf

We need to aggregate the transactions from the incoming stream, the `InputRow` case class, by the `CustomerID` field. *Figure 5.11* shows an example table of the incoming stream. Additionally, we must remove transactions once they have fallen outside the specified window; we will call these "expired transactions." This means we must create and maintain a state for each customer, the `TransactionCountState` case class. Each customer state consists of `CustomerID`, `transactionCount`, and `transactionList` containing the times that the transactions occurred for each transaction accounted for in the transaction count. *Figure 5.12* is a visual representation of the customer state. As a customer transaction arrives, it is added to a list of transactions with a timestamp as part of the customer state.

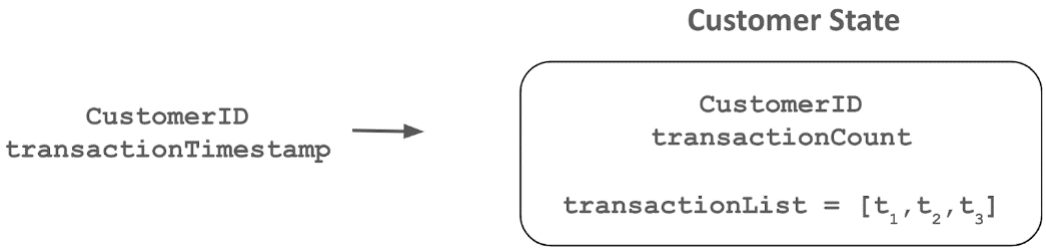


Figure 5.7 – Each transaction updates the customer state

The state is created by applying the stateful streaming logic to the input data. The customer state is then used to write the feature table, `transaction_count_ft`, shown in *Figure 5.8*: